

复习

第1章 软件工程学概述

第2章 可行性分析

第3章 需求分析

第5章 总体设计

第6章 详细设计

第7章 实现

第8章 维护

第9章 面向对象方法学引论

第10章 面向对象分析

第11章 面向对象设计

第1章 软件工程学概述

◆软件危机

- **定义：** 指在计算机软件的**开发和维护过程**中所遇到的一系列严重问题。
- **典型表现：**
 - 开发成本和进度的估计常常很不准确；
 - 用户对“已完成的”软件系统不满意；“闭门造车”；
 - 软件质量不可靠；
 - 软件常常是不可维护的；
 - 软件成本的比例逐年上升；
 - 软件产品“供不应求”；
- **产生软件危机的原因**
- 一方面与软件本身的特点有关，另一方面也和软件开发与维护的方法不正确有关。
 - 软件的规模加大、复杂性提高、性能增强；
 - 软件是逻辑产品：软件缺乏“可见性”，不会“用坏/磨损”，软件产品的成本主要体现在软件的开发和维护上；
 - 缺乏系统的开发、维护大型软件项目的技术手段和管理方法；
 - 用户和软件开发人员的理解鸿沟；

软件危机

消除软件危机的途径

- 消除“软件就是程序”的错误观念
软件是程序、数据及相关文档的完整集合。文档是开发、使用和维护程序所需要的图文资料。
- 软件开发是工程项目
- 成功的软件开发技术和方法
- 软件工具和软件工程支撑环境

第1章 软件工程学概述

◆ 1968年 NATO 计算机科学会议

◆ 软件危机 □ 解决途径 □ 软件工程

◆ 软件工程

● 定义

➤ 软件工程是指导计算机软件**开发和维护**的一门**工程学科**。采用工程的概念、原理、技术和方法来开发与维护软件，把经过时间考验而证明正确的**管理技术**和当前能够得到的最好的技术方法结合起来，以**经济**地开发出**高质量**的软件并**有效地维护**它。

● 本质特性

➤ 软件工程关注于大型程序的构造

➤ 软件工程的中心课题是控制复杂性

➤ 软件经常变化

➤ 开发软件的效率非常重要

➤ 和谐地合作是开发软件的关键

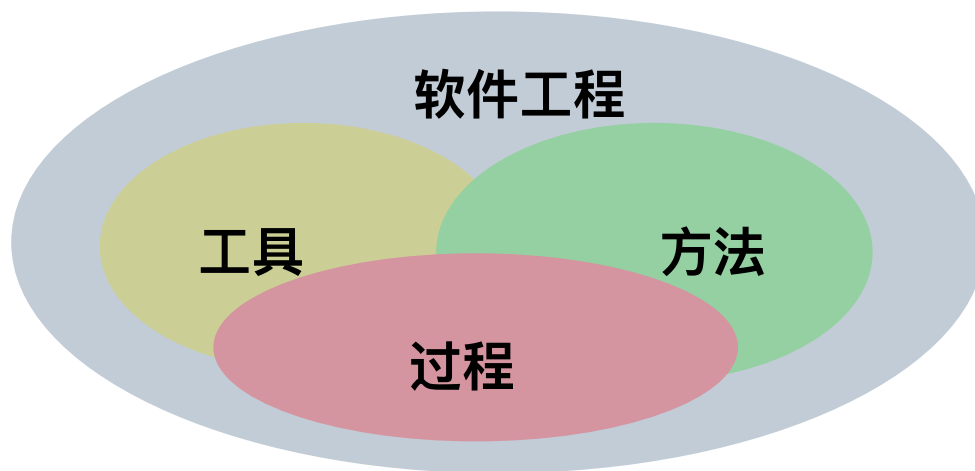
➤ 软件必须有效地支持它的用户

➤ 在软件工程领域中通常由具有一种文化背景的人替具有另一种文

第1章 软件工程学概述

软件工程方法学

- 包括技术和管理两方面
- 3要素



第1章 软件工程学概述

- 软件生命周期是软件产品或系统一系列相关活动的全周期。
- 软件生命周期的3个大阶段
 1. **软件定义**: 确定软件开发总目标; 确定工程的可行性; 导出实现策略及系统功能; 估计资源和成本, 并且制定工程进度表。
 - 问题定义、可行性研究、需求分析
 2. **软件开发**: 具体设计和实现在前一个时期定义的软件
 - 总体设计、详细设计、编码和单元测试、综合测试
 3. **软件维护**: 使软件持久地满足用户的需要

第1章 软件工程学概述

◆ 生命周期中各阶段的任务

1. **问题定义**：“要解决的问题是什么？”； 确定用户要求解决的性质、工程的目标和规模。
2. **可行性研究**：“对于上一个阶段所确定的问题有行得通的解决办法吗？”，经济可行性、技术可行性、法律可行性、不同的方案
3. **需求分析**：“为了解决这个问题，目标系统必须做什么”， 确定系统必须具有的功能和性能，系统要求的运行环境，并且预测系统发展的前景。
4. **总体设计（概要设计）**：“概括地说，应该怎样实现目标系统？”； 设计出实现目标系统的几种可能的方案。推荐一个最佳方案。
5. **详细设计**：“应该怎样具体地实现这个系统呢？”； 设计出程序的详细规格说明。
6. **编码和单元测试**： 写出正确的容易理解、容易维护的程序模块； 仔细测试编写出的每一个模块。
7. **综合测试**： 集成测试和验收测试，现场测试或平行运行
8. **软件维护**： 使系统持久地满足用户的需要。包括： 改正性维护，适应性维护，完善性维护，预防性维护。

第1章 软件工程学概述

生命周期模型规定了把**生命周期划分成哪些阶段及各个阶段的执行顺序**，因此，也称为过程模型。

● 典型的过程模型

1. 瀑布模型(Waterfall model)
2. 快速原型开发模型(Rapid Prototyping model)
3. 增量模型(Incremental model)
4. 螺旋模型(Spiral model)
5. 喷泉模型

第1章 软件工程学概述

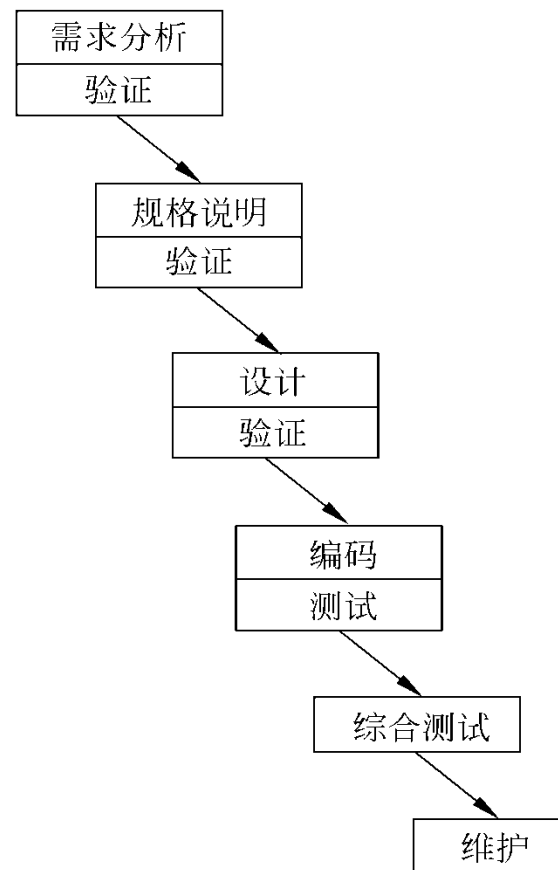
瀑布模型

特点

1. 阶段间具有顺序性和依赖性
2. 推迟实现的观点
— 清楚地区分逻辑设计与物理设计，
— 尽可能推迟程序的物理实现。
3. 质量保证的观点(文档驱动)
— 每个阶段都必须完成规定的文档
— 每个阶段结束前都要对所完成的文档进行评审

缺点

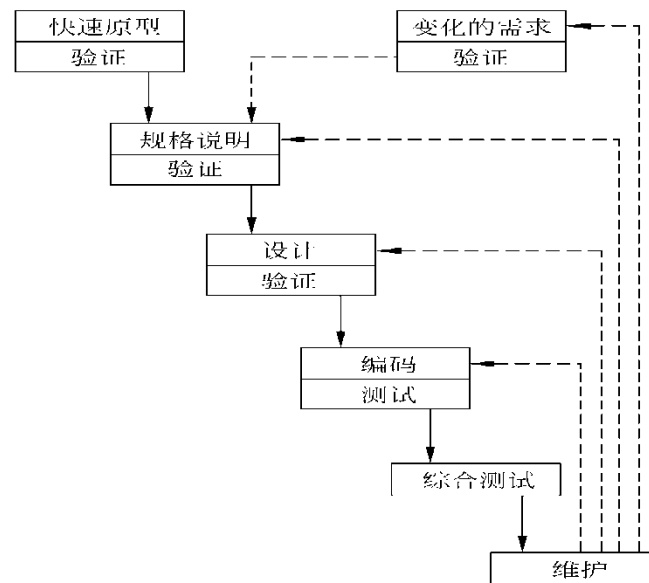
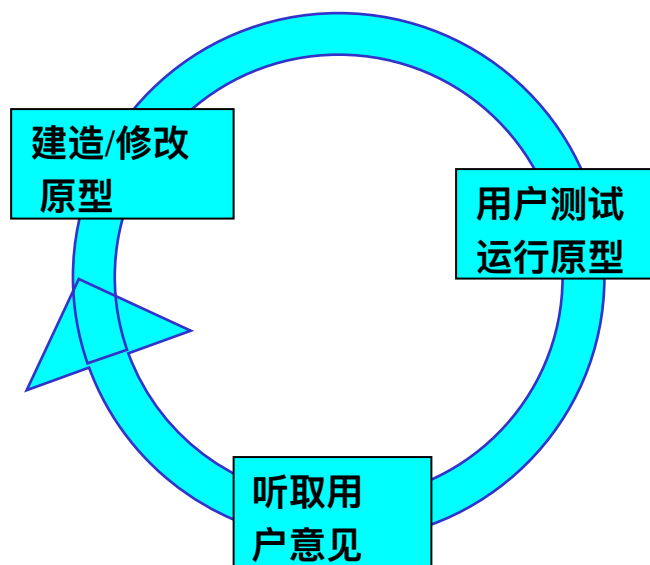
4. 开发过程一般不能逆转，否则代价太大。
5. 规格说明很难理解：“我知道这是按我的要求做的，但不是我想要的样子。”
6. 软件的实际情况必须到项目开发的后期客户才能看到。



第1章 软件工程学概述

快速原型模型

- **特点:**快速原型的本质是“快速”;快速原型可以取代规格说明阶段,但不是设计阶段,容易适应需求的变化;有利于开发与培训的同步
- **应用范围:**用户需求不完全或不准确;项目招投标时;产品移植或升级或对已有产品原型进行客户化工作



第1章 软件工程学概述

◆ 增量模型

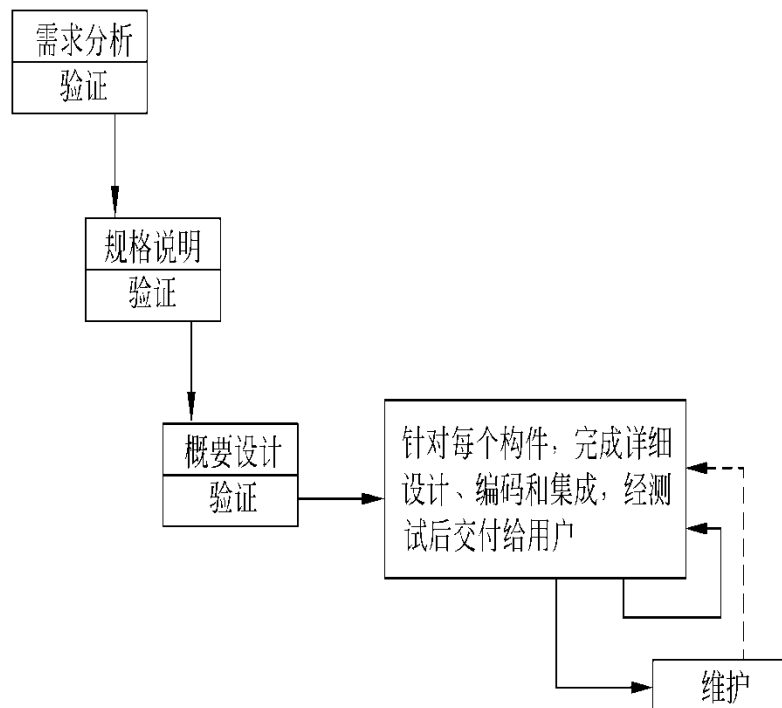
- 把软件产品作为一系列增量构件来设计、编码、集成和测试。

● 优点

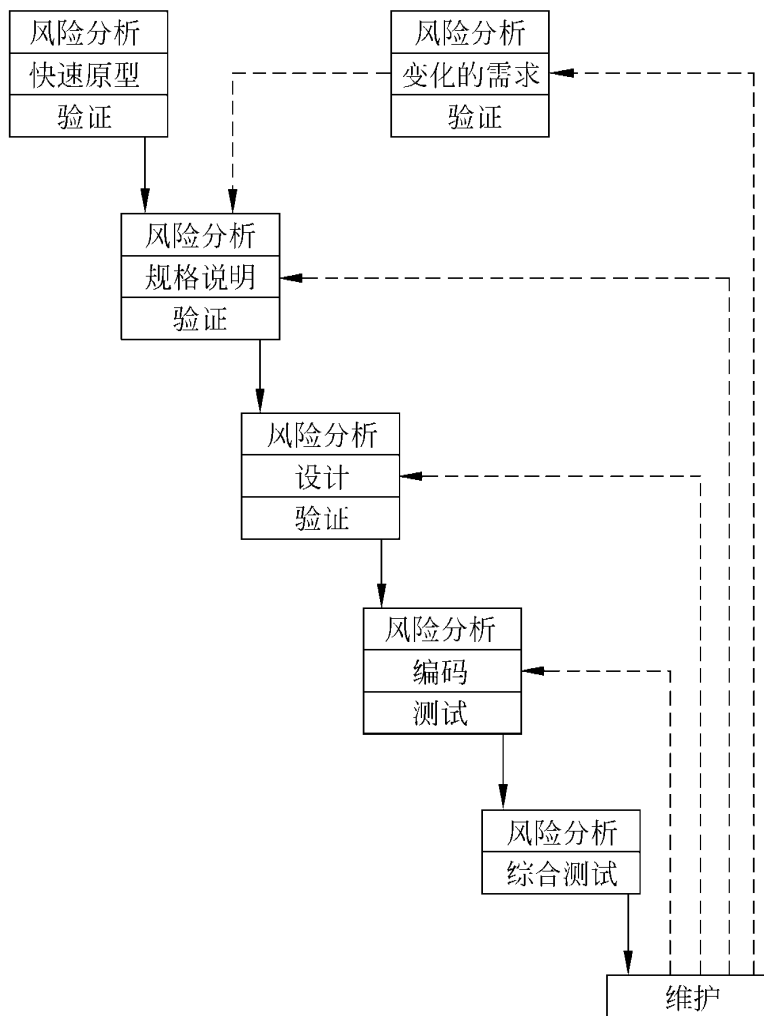
- 每个阶段交付一个可用的产品。
- 减少一个全新产品给客户带来的心理上的影响。
- 分阶段地交付产品不需要大的资金支出。
- 需求经常变化，增量模型的灵活性使其具有更加优越的适用性。

● 缺点

- 需要一个开放的结构，方便构件的加入



简化的螺旋模型...



简化版本：瀑布模型和快速原型模型的有机结合+风险分析

每个阶段之前

确定目标，可供选择的办法
及其限制条件+风险分析

每个阶段之后

评估并计划下一阶段

螺旋模型的优点

容易确定何时已经对某一阶段的产品充分测试完毕

维护和开发之间没有什么本质上的差别

螺旋模型的缺点

螺旋模型的特点

◆ 螺旋模型的优点

- 容易确定何时已经对某一阶段的产品充分测试完毕
- 维护和开发之间没有什么本质上的差别

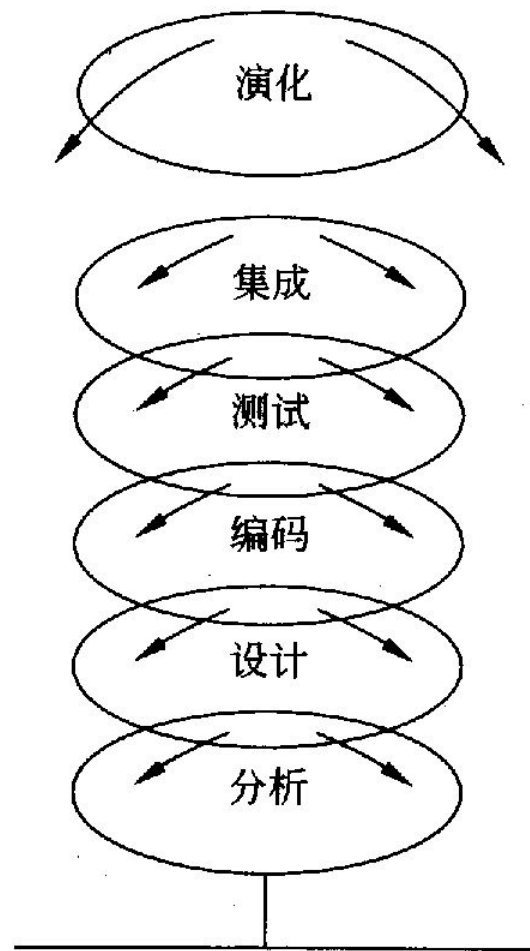
◆ 螺旋模型的缺点

- 仅适合于大型软件

◆ 风险驱动既是优点也是缺点

喷泉模型

- ◆ 喷泉模型是一种支持面向对象开发的模型
- ◆ 体现迭代和无间隙特征
 - **迭代**：各开发活动常常重复工作多次，相关的功能在每次迭代中随之加入演进的系统
 - **无间隙**：开发活动之间不存在明显的边界



基于构件的开发模型

- ◆ 支持软件**复用** (reuse)
- ◆ 利用预先包装好的软件**构件** (包括组织内部开发的构件和现存商品化构件COTS) 来构造应用系统

RUP

◆ RUP 软件开发生命周期是一个二维的生命周期模型，RUP 把软件生命周期划分成 4 个连续的阶段。

- 初始阶段：建立业务模型，定义最终产品视图，并且确定项目的范围。
- 精化阶段：设计并确定系统的体系结构，制定项目计划，确定资源需求。
- 构建阶段：开发出所有构件和应用程序，把它们集成为客户需要的产品，并且尽地测试所有功能。
- 移交阶段：把开发出的产品提交给用户使用。

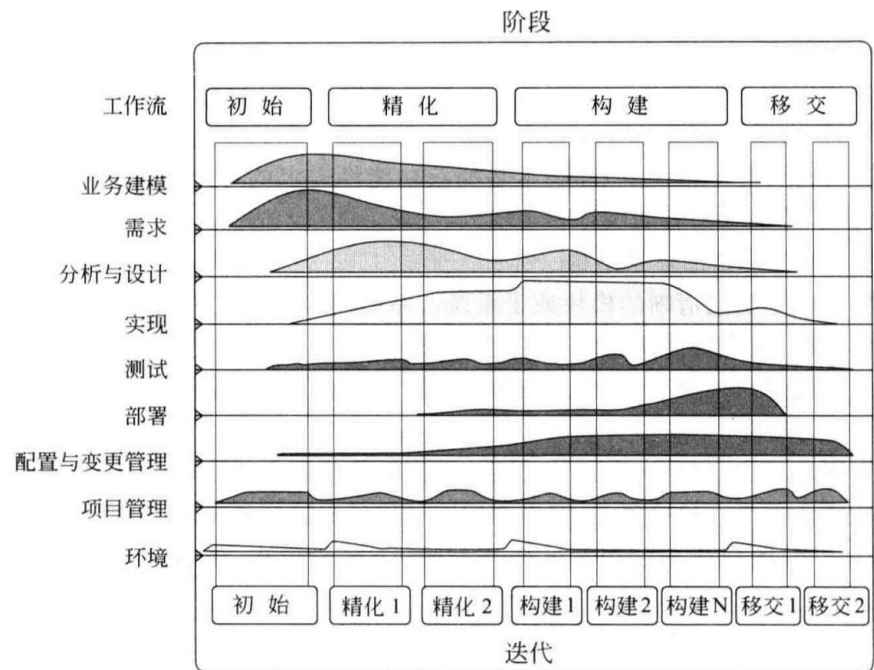


图 1.10 RUP 软件开发生命周期

RUP

◆ 优缺点与适用范围

优点：

- ①允许变更需求
- ②逐步集成元素
- ③尽早降低风险
- ④有助于提高团队士气和生产力
- ⑤生成更高质量的产品
- ⑥保证项目开发进度
- ⑦容许产品进行战术改变
- ⑧迭代流程自身可在进行过程中得到改进和精炼

适用：

大型的系统开发、面向对象系统的开发

缺点：

RUP只是一个开发过程，并没有涵盖软件过程的全部内容，例如它缺少关于软件运行和支持等方面的内容；此外，它没有支持多项目的开发结构，这在一定程度上降低了在开发组织内大范围实现重用的可能性。

敏捷开发和极限编程

◆ 所谓敏捷开发(Agile)，就是一种面临迅速变化的需求快速开发软件的能力

- 使用一些**实践**，以提供必要的纪律和反馈
- 使用一些**设计原则**，以保持软件是灵活的、可维护的
- 知道一些**设计模式**，以便针对特定问题权衡这些原则

◆ 敏捷(Agile)联盟宣言

- 个体和交互 **胜过** 过程和工具
- 可以工作的软件 **胜过** 面面俱到的文档
- 客户合作 **胜过** 合同谈判
- 响应变化 **胜过** 遵循计划

◆ 具有对变化和不确定性的更快速、更敏捷的反应特性，而且在快速的同时仍然能够保持可持续的开发速度。

极限编程

- ◆ **极限编程(XP)(由Kent Beck提出)是在增量模型基础上发展起来的一种颇有争议的新的软件开发方法**
 - **快速反馈：**团队成员理解客户在开发过程中给出的反馈，并能够迅速做出反应。
 - **假设简单：**开发人员需要专注于当前重要的工作，遵循所有问题都可以简单地解决原则。
 - **增量变化：**对产品一小步一小步的改变比一次性的大改变效果更好。
 - **拥抱变化：**如果客户认为产品需要改变，作为程序员，他们应该支持这个决定，并拟定如何实现新需求的计划。
 - **高质量的工作：**一个工作出色的团队，他们能够创造出有价值的产品，并为此感到自豪

生命周期模型		优点	缺点	适用范围
瀑布模型	文档驱动的有序方法 简单、易用、直观	交付产品可能不符合客户的要求 不允许变更或者限制变更	在开发中，向下、渐进的路径占据支配地位，即要求在项目开始前，项目的需求已经被很好理解，也很明确；而且项目经理很熟悉为实现这一模型所需要的过程，同时解决方案在项目开始前也很明确。类似的项目如公司的财务系统，库存管理系统及短期项目。一个新的项目不适合瀑布模型，除非处于项目开始之前项目的需求不明确，或者需要减少项目的不确定性的时候，可以采用原型方法。类似的项目如：需要明确系统的界面，验证一些技术的可行性等。	
快速原型模型	确保交付的产品符合客户的要求	还没有证明无懈可击	项目开始时明确了大部分的需求，但是需求可能会发生变化的项目；对于市场和用户把握不是很准，需要逐步了解的项目；对于有庞大和复杂功能的系统进行功能分解时，需要逐步明确需求和风险限制了项目进度的项目；用户对自己的需求不是很明确的项目；需要对一些基本的概念进行验证的项目；可能发生一些重大变更的项目；规模很大的项目；采用了新技术的项目。	
增量模型	增大投资的早期回报	要求开放的结构，可能退化为建造-修补模型 能用于大型的内部软件产品，开发者必须精通风险分析和风险评估还没有被广泛使用		
螺旋模型	结合上述所有模型的特性			
极限编程	增大投资的早期回报，客户的需求模糊时			

第2章 可行性分析

可行性分析的任务

1. 目的不是要解决问题，而是确定问题是否值得去解决
2. 用最小的代价在尽可能短的时间内确定问题是否能够解决
3. 最根本的任务是对以后的行动方针提出**建议**

三个可行性

- (1) 技术可行性：使用现有的技术能实现这个系统吗？
 - (2) 经济可行性：这个系统的经济效益能超过它的开发成本吗？
 - (3) 操作可行性：系统的操作方式在这个用户组织内行得通吗？
- 必要时还应该从法律、社会效益等更广泛的方面研究每种解法的可行性。

研究过程

系统流程图

- 概括地描绘物理系统的传统工具
- 基本思想是用图形符号以黑盒子形式描绘组成系统的每个部件（程序、文档、数据库、人工过程等
- 表达的是数据在系统各部件之间流动的情况，而不是对数据进行加工处理的控制过程
- 是物理数据流图而不是程序流程图

第3章 需求分析

需求分析的任务

1. 准确地回答“系统必须做什么?”

2. “分析软件需求和书写软件需求规格说明书”

软件需求

- 用户解决问题或达到目标所需要的条件或能力;

- 系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或能力

- 反映上述两个定义中所描述的条件或能力的文档说明

- 需求层次: 业务需求 □ 用户需求 □ 功能与非功能需求

具体任务

- 确定对系统的综合要求

- 功能需求、性能需求、可靠性和可用性需求、出错处理需求、接口需求、约束、逆向需求、扩展需求

- 分析系统的数据要求

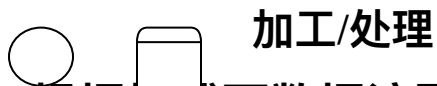
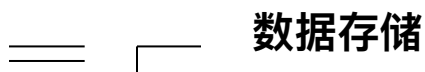
- 导出系统的逻辑模型

- 修正系统开发计划

第3章 需求分析

◆ 数据流图

- 数据流图 (DFD) 符号



- 根据描述画数据流图

◆ 状态图

- 通过描绘系统的状态及引起系统状态转换的事件，来表示系统的行为。

➤ 状态

➤ 事件

➤ 行为

- 画状态图

第5章 总体设计 (概要设计)



总体设计的任务

1. “概括地说，系统应该如何实现？”
2. 系统划分:即确定组成系统的程序、文件、数据库、人工过程和文档等
3. 设计软件的结构: 即确定每个程序是由哪些模块组成, 以及这些模块相互间的关系。

第5章 总体设计

设计原理

1. 模块化

模块化的根据

$$C(P1+P2) > C(P1) + C(P2)$$

$$E(P1+P2) > E(P1) + E(P2)$$

2. 抽象

抽象就是抽出事物的本质特性而暂不考虑它们的细节

3. 逐步求精

为了能集中精力解决主要问题而尽量推迟对问题细节的考虑。

4. 信息隐藏和局部化

信息隐藏原理：应该这样设计和确定模块，使得一个模块内包含的信息(过程和数据)对于不需要这些信息的模块来说，是不能访问的。

局部化：把一些关系密切的软件元素物理地放得彼此靠近。

5. 模块独立

每个模块完成一个相对独立的子功能，并且与其它模块间的接口简单。

模块独立性的衡量标准

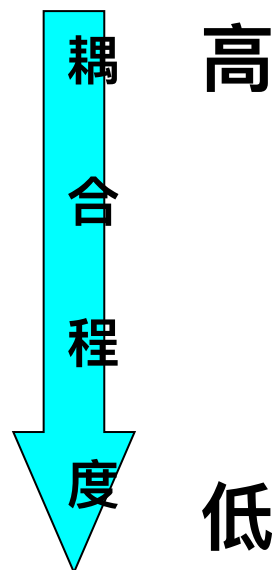
模块内聚 (Cohension)：模块内各元素交互的程度

模块耦合 (Coupling)：模块间交互程度

第5章 总体设计

◆ 耦合

1. 内容耦合
2. 共用耦合
3. 控制耦合
4. 印记（特征）耦合
5. 数据耦合



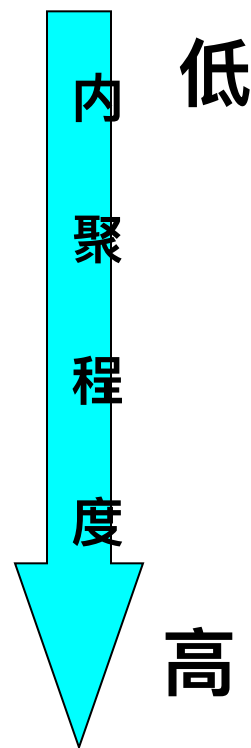
◆ 各种耦合的含义

◆ 设计时力争做到低耦合。应该采取的设计原则是：尽量使用数据耦合，少用控制耦合和特征耦合，限制公用耦合的范围，完全不用内容耦合。

第5章 总体设计

◆ 内聚

1. 偶然性内聚
2. 逻辑性内聚
3. 时间性内聚
4. 过程性内聚
5. 通信性内聚
6. 顺序内聚
7. 功能性内聚
8. 信息性内聚



◆ 各种内聚的含义

◆ 设计时力争做到高内聚，并且能够辨认出低内聚的模块，通过修改设计提高模块的内聚程度。

第5章 总体设计



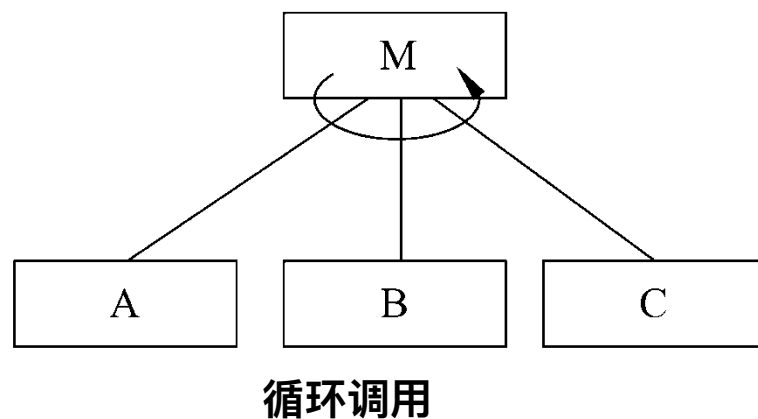
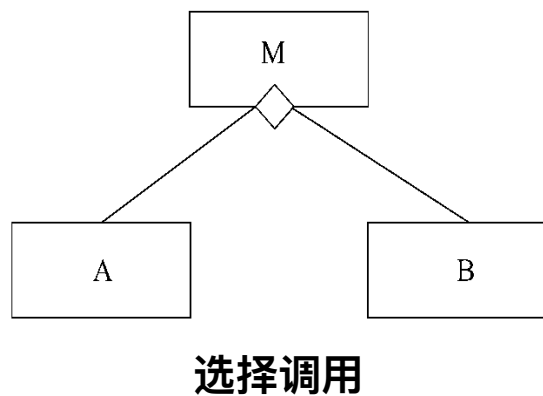
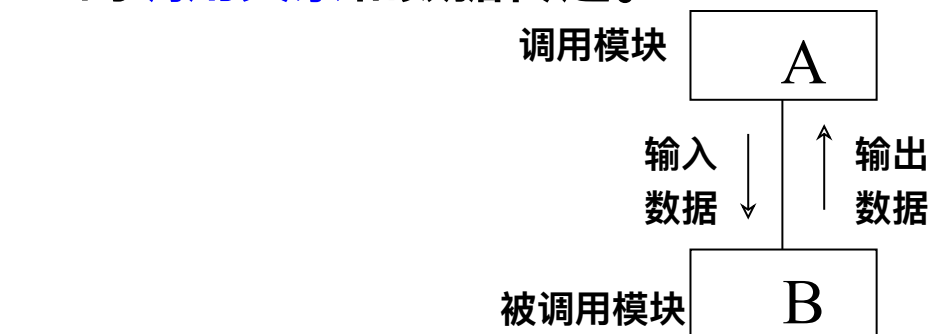
启发规则

1. 改进软件结构提高模块独立性
2. 模块规模应该适中
3. 深度、宽度、扇出和扇入都应适当
4. 模块的作用域应该在控制域之内
 - 模块的作用域：受该模块内一个判定影响的所有模块的集合。
 - 模块的控制域：模块本身以及所有直接或间接从属于它的模块的集合。
 - 所有受判定影响的模块应该都从属于做出判定的那个模块，最好局限于做出判定的那个模块本身及它的直属下级模块。
5. 力争降低模块接口的复杂程度
6. 设计单入口单出口的模块
7. 模块功能应该可以预测

第5章 总体设计

◆ 结构图

- 描述软件系统的模块层次结构，清楚地反映出程序中各模块之间的调用关系和数据传递。



第5章 总体设计



面向数据流的设计方法



基本思想： DFD □ 结构图



两种信息流类型



变换流



事务流



变换流到软件结构图的转换

第6章 详细设计

◆ 详细设计的任务

- 不是具体地编写程序，而是设计程序的“蓝图”，确定每个模块的处理过程。

◆ 结构程序设计

- 结构化定理
 - 任何单入口单出口的程序都可以由“顺序”、“选择”和“循环”三种基本结构实现。
- 过程设计的工具（重点是画图）
 - 程序流程图
 - 盒图 (NS图)
 - PAD图
 - 判定表、树

第7章 实现

编码风格

- 编码风格的作用就是使代码容易读；
- 风格良好的代码更容易阅读和理解，错误更少；
 1. 使用一致和有意义的标识符名
 2. 用缩进显示程序结构
 3. 用加括号的方式排除二义性
 4. 避免大量使用循环嵌套和条件嵌套
 5. 当心运算符的副作用
 6. 把数定义称常量
 7. 利用sizeof()计算对象的大小
 8. 清晰的代码，而非最巧妙的代码
 9. 程序的注释
 - 序言性注释和功能性注释
 - 对一段程序注释，而不是每一个语句
 10. 使用数据结束标记（EOF、BOF），不要指定数据的数目来判断文件的结束。

第7章 实现

测试

- **测试的目的**就是在软件投入生产性运行之前，尽可能多地发现软件中的错误。

- **调试的目的**是诊断并改正错误，由程序员完成。

- 对软件规格说明、设计和编码的最后复审。

- 开发总工作量的40%以上，极端情况下，其他开发步骤总成本的3倍到5倍。

- 测试是为了发现程序中的错误而执行程序的过程。

- 好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案。

- 成功的测试是发现了至今为止尚未发现的错误的测试。

- 测试只能查找出程序中的错误，不能证明程序中没有错误。

- Pareto原理：80%的错误很可能是20%的模块造成的。

- 从“小规模”测试逐步到“大规模”测试。

- 穷举测试是不可能的。

- 为了达到最佳的测试效果，应该由独立的第三方从事测试工作。

第7章 实现

测试方法

- 黑盒测试：又称功能测试或数据驱动测试

- 白盒测试：又称结构测试或逻辑驱动测试

测试步骤

- 模块测试（单元测试）

- 在这个测试步骤中所发现的往往是编码和详细设计的错误

- 子系统测试

- 模块放在一起形成一个子系统来测试

- 着重测试模块的接口

- 系统测试

- 经过测试的子系统装配成一个完整的系统来测试

集成测试

- 发现的往往是软件设计中的错误，也可能发现需求说明中的错误

- 验收测试（确认测试）

- 它的目标是验证软件的有效性（如果软件的功能和性能如同用户所合理期待的那样，软件就是有效的）

- 用户积极参与，可能主要使用实际数据进行测试

- 发现的往往是系统需求说明书中的错误

- 平行运行

第7章 实现

◆ 回归测试

- 回归测试是指重新执行已经做过的测试的某个子集，以保证变化（程序改错、新模块加入等）没有带来非预期的副作用。

◆ (□) Alpha测试

在开发者的“指导”下，由用户在开发场所进行。

◆ (□) Beta测试

多个用户在实际使用环境下进行测试。

第7章 实现

◆ 集成测试方法

- 非渐增式集成

- 先分别测试每个模块，再把所有模块按设计要求放在一起结合成所要的程序。
- 先进行单元测试，再进行集成测试

- 渐增式集成

将单元测试与集成测试结合在一起，把下一个要测试的模块同已经测试好的那些模块结合起来进行测试，测试完以后再把下一个应该测试的模块结合进来测试。

需要存根程序、驱动程序

- 自顶向下 (Top-Down) 集成
- 自底往上 (Bottom-Up) 集成
- 三明治式 (Sandwich) 集成

第7章 实现

◆ 几种集成测试方法的优缺点

方法	优点	不足
非渐增式集成		没有错误隔离手段 主要设计错误发现迟
自顶向下集成	错误隔离 较早发现主要设计错误	可复用模块得不到充分测试
自底往上集成	错误隔离 可复用模块得到充分测试	主要设计错误发现迟
三明治式集成	错误隔离 较早发现主要设计错误 可复用模块得到充分测试	

第7章 实现

◆ 白盒测试技术：逻辑覆盖

- 定义：以程序内部的逻辑结构为基础的设计测试用例的技术
- 类型

- 语句覆盖：选择足够多的测试数据，使被测程序中每个语句至少执行一次。

- 判定覆盖：判定覆盖又叫分支覆盖，选择足够多的测试数据使每个判定的每个分支都至少执行一次。

- 条件覆盖：选择足够多的测试数据使每个判定表达式中的每个条件都取到各种可能的结果

- 判定 / 条件覆盖：选取足够多的测试数据，使得判定表达式中的每个条件都取到各种可能的值，而且每个判定表达式也都取到各种可能的结果。

- 条件组合覆盖：选取足够多的测试数据，使得每个判定表达式中条件的各种可能组合都至少出现一次。

- 设计测试用例（会做）

第7章 实现



黑盒测试技术

1.

等价划分



把程序的输入域划分成若干个**数据类**，每类中的一个典型值在测试中的作用与这一类中所有其他值的作用相同。据此导出测试用例。



设计测试用例

- (1) 设计一个新的测试方案以尽可能多地覆盖尚未被覆盖的有效等价类，重复这一步骤直到所有有效等价类都被覆盖为止；
- (2) 设计一个新的测试方案，使它覆盖一个而且只覆盖一个尚未被覆盖的无效等价类，重复这一步骤直到所有无效等价类都被覆盖为止。

2.

边界值分析



着重测试输入等价类和输出等价类的边界，选取的测试数据应该刚好等于、刚刚小于和刚刚大于边界值。



会用等价划分和边界值分析法设计测试用例

第8章 维护

- ◆ 所谓软件维护就是在软件已经交付使用之后，为了改正错误或满足新的需要而修改软件的过程。保证软件在一个相当长的时期能够正常运行。
- ◆ 60%以上, 这个百分比还在持续上升。
- ◆ 维护类型
 - 改正性维护
诊断和改正错误的过程。17%~21%
 - 适应性维护
为了适应环境的变化进行的修改软件的活动。18%~25%
 - 完(改)善性维护
增加新功能或修改已有功能。50%~66%
 - 预防性维护
为了改进未来的可维护性或可靠性，或为了给未来的改进奠定更好的基础而修改软件。4%左右

软件的可维护性

- ◆ 维护人员理解、改正、改动或改进这个软件的难易程度。
- ◆ 提高可维护性是支配软件工程方法学所有步骤的关键目标。
- ◆ 决定软件可维护性的因素
 - 1) 可理解性
 - 2) 可测试性
 - 3) 可修改性
 - 4) 可移植性
 - 5) 可重用性

第9章 面向对象方法学引论

◆ 面向对象方法学的4个要点

- 1) 客观世界是由各种**对象 (Object)**组成。面向对象的软件系统是由对象组成的。
- 2) 对象组成对象**类(Class)**。类是具有相同属性和行为的对象的集合。每个对象类定义了一组数据和一组方法。
- 3) 按照子类(派生类)与父类(基类)的关系，对象类组成一个层次结构的系统(类等级)。子类**继承(inheritance)**父类的数据和方法。
- 4) 对象彼此之间仅能通过传递**消息(Message)**互相联系。
OO=objects+classes+inheritance+communication with messages

面向对象 = 对象 + 类 + 继承 + 消息通信

第9章 面向对象方法学引论

◆ 面向对象的一些概念

- **对象**：封装了数据结构及可以施加在这些数据结构上的操作（**服务或方法**）的封装体。特点：以数据为中心、对象是主动的、实现了数据封装、本质上具有并行性、模块独立性好。
- **类**：具有相同数据和相同操作的一组相似对象的集合
- **消息**：要求某个对象执行在定义它的那个类中所定义的某个操作的规格说明。
- **方法**：对象所能执行的操作，也就是类中所定义的服务。（如C++的成员函数）
- **属性**：类中所定义的数据（如C++的数据成员）

第9章 面向对象方法学引论

◆ 面向对象的一些概念

- **封装**：表示对象状态的数据和实现操作的代码与局部数据，都被封装在黑盒子里面，不能从外面直接访问或修改这些数据和代码。封装也就是信息隐藏。
- **继承**：是子类自动地共享基类中定义的数据和方法的机制。继承性使得相似的对象可以共享程序代码和数据结构，从而大大减少了程序中的冗余信息。
- **多态性**：同一方法，不同的子类有不同的实现。多态性机制不仅增加了面向对象软件系统的灵活性，进一步减少了信息冗余，而且显著提高了软件的可重用性和可扩充性。
- **重载**：是指在同一作用域内的若干个参数特征不同的函数可以使用相同的函数名字。重载进一步提高了面向对象系统的灵活性和可读性。

第9, 10章 面向对象方法学

◆ 面向对象建模

● 三种模型

- **对象模型**：它是对模拟客观世界实体的对象以及对象彼此间的关系的映射，描述系统的数据结构——对谁做——类图
- **动态模型**：描述系统的控制结构——何时做——事件跟踪图(顺序图)、状态图
- **功能模型**：描述系统的功能——做什么——用例图、数据流图

● 建模图形工具

- **类图**：类图描述类及类之间的静态关系（关联、聚集、泛化等）。（根据描述画图）
- **类的状态图**：通过描绘对象的状态及引起对象状态转换的事件，来表示对象的行为。（根据事件跟踪图画状态图）
- **用例图**：描述的是外部行为者所理解的系统功能。系统、行为者、用例及用例之间的关系。（根据描述画图）
- **事件跟踪图**：（根据描述画图）

第11章 面向对象设计

◆ 面向对象设计的任务

- **系统设计**: 确定实现系统的策略和目标系统的高层结构;
- **对象设计**: 确定解空间中的类、关联、接口形式及实现服务的算法。

第11章 面向对象设计

◆ 面向对象设计的准则

- 模块化、抽象、信息隐藏

- 耦合

- (1) 交互耦合

- 对象之间的耦合通过消息连接来实现，则这种耦合就是交互耦合。应该尽量减少消息中包含的参数个数，降低参数的复杂程度。减少对象发送(或接收)的消息数。

- (2) 继承耦合

- 继承是一般化类与特殊类之间耦合的一种形式。设计应该使特殊类尽量多继承并使用其一般化类的属性和服务。

- 内聚

- (1) 服务内聚

- 一个服务应该完成一个且仅完成一个功能。

- (2) 类内聚

- 类的属性和服务应该全都是完成该类对象的任务所必需的，其中不包含无用的属性或服务。

- (3) 一般-特殊内聚

- 一般-特殊结构应该是对相应的领域知识的正确抽取。

第11章 面向对象设计

◆ 重用

- 重用有两方面的含义：一是尽量使用已有的类，二是如果确实需要创建新类，则在设计这些新类的协议时，应该考虑将来的可重复使用性。
- 重用的3个层次：知识、方法和标准、软件成分
- 软件成分的重用级别：代码重用、设计结果重用、分析结果重用
- 类构件的重用方式
 - (1) 实例重用
 - 按照需要创建类的实例
 - 用几个简单的对象作为类的成员创建一个更复杂的类。
 - (2) 继承重用
 - 为提高继承重用的效果，关键是设计一个合理的、具有一定深度的类构件继承层次结构。
 - (3) 多态重用
 - 利用多态性不仅可以使对象的对外接口更加一般化，从而降低了消息连接的复杂程度，而且还提供了一种简便可靠的软构件组合机制。

第11章 面向对象设计

◆ 子系统间的交互方式

(1) 客户-供应商关系：单向交互

“客户”子系统调用“供应商”子系统，“供应商”子系统完成某些服务工作并返回结果。

(2) 平等伙伴关系：双向交互

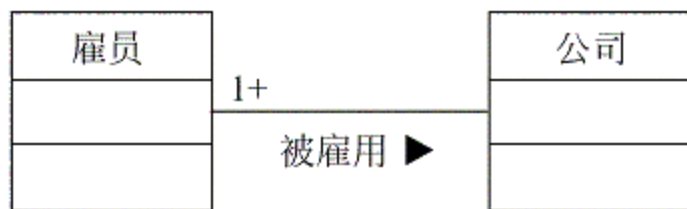
每个子系统都可能调用其他子系统，因此，每个子系统都必须了解其他子系统的接口。

第11章 面向对象设计

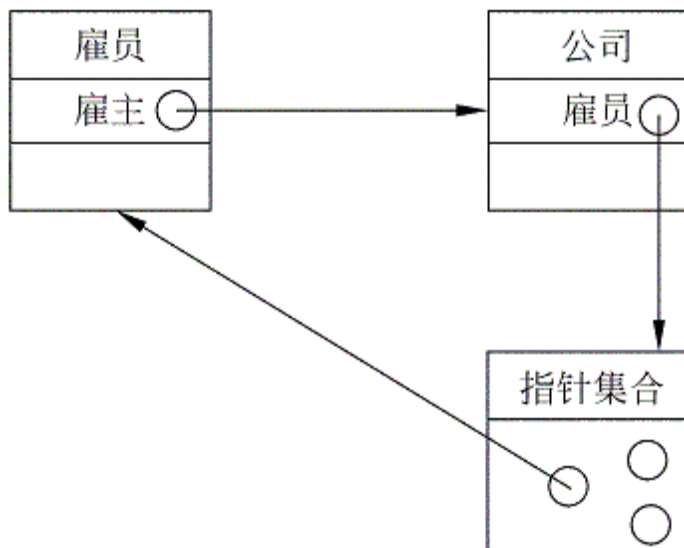
◆ 设计关联

- 实现关联的具体策略
 - 单向遍历
 - 双向遍历
 - 用属性实现两个方向的关联
 - 用独立的关联对象实现双向关联

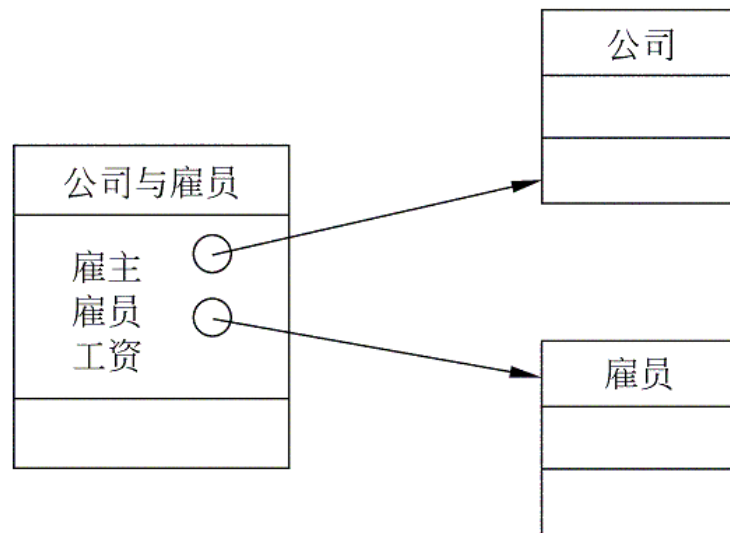
第11章 面向对象设计



1, 用属性实现一个方向的关联



2, 用属性实现两个方向的关联



3, 用独立的关联对象实现双向关联

【说明】某图书管理系统的主要功能如下：

1. 图书管理系统的资源目录中记录着所有可供读者借阅的资源，每个资源项都有唯一的索引号。系统需登记每项资源的名称、出版时间和资源状态（在库或已借出）。
2. 资源可以分为两类：图书和唱片。对于图书，系统还需登记作者和页数；对于唱片，还需登记演唱者和介质类型（CD或者磁带）。
3. 读者信息保存在图书管理系统的读者信息数据库中，记录的信息包括：读者的识别码和读者姓名。系统为每个读者创建了一个借书记录文件，用来保存读者所借资源的相关信息。

上述说明中带下划线的名字表示找出的类，请用类图描述这些类之间的联系

第11章 面向对象设计

